



ITI · PYTHON FRAMEWORK WITH FASTAPI TRACK

Python Framework with FastAPI

Eng. Fatma Elraey*Lab 3 — Simple ORM Persistence with SQLite*

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lab 3 of 5
Focus	Practice SQLite persistence with a simple SQLAlchemy ORM model, direct Session(engine), and basic CRUD operations.
Evaluation	Lab Assignment — 100%

Before You Start

This lab continues the same Documents API from Lectures 1 and 2. In this version, keep the database work intentionally simple: one SQLite file, one SQLAlchemy engine, one ORM model, and one Session(engine) block inside each endpoint. Build and test one step at a time.

Today's Lab Plan

1. Install SQLAlchemy and prepare the existing FastAPI project
2. Connect SQLAlchemy to the SQLite file documents.db
3. Define the Document ORM model and create the table
4. Read and create documents with Session(engine)
5. Update and delete persisted documents
6. Restart the API and verify that the data is still stored

Install

```
pip install sqlalchemy
```

01 SQLite Engine & Base

Goal: Connect SQLAlchemy to one local SQLite database file and create the Base class for ORM models.

Steps

1. Keep this beginner lab in one main.py file.
2. Import create_engine and DeclarativeBase.
3. Create an engine for sqlite:///documents.db.
4. Define Base as the parent class for ORM models.

Starter / reference code

```
# main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase

app = FastAPI()

engine = create_engine("sqlite:///documents.db")

class Base(DeclarativeBase):
    pass
```

✓ CHECKPOINT — Exercise 1

Q1. What does the engine point SQLAlchemy to?

02 Define the Document ORM Model

Goal: Map a Python Document class to the documents database table.

Steps

1. Import String, Text, Mapped and mapped_column.
2. Define id, title, content and priority columns.
3. Set id as the primary key and priority default to 1.
4. Call Base.metadata.create_all(engine) so the table is created when missing.

```
from sqlalchemy import String, Text
from sqlalchemy.orm import Mapped, mapped_column

class Document(Base):
    __tablename__ = "documents"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(120))
    content: Mapped[str] = mapped_column(Text)
    priority: Mapped[int] = mapped_column(default=1)

Base.metadata.create_all(engine)
```

✓ CHECKPOINT — Exercise 2

Q1. What does create_all(engine) do in this lab?

03 Read with Session(engine)

Goal: Open a simple SQLAlchemy Session directly inside an endpoint and read persisted rows.

Steps

1. Import Session from sqlalchemy.orm.
2. Use with Session(engine) as db: inside the route.
3. Query the Document model and return all rows.
4. Leave the with block and let the Session close automatically.

Starter / reference code

```
from sqlalchemy.orm import Session

@app.get("/documents")
def list_documents():
    with Session(engine) as db:
        return db.query(Document).all()
```

✓ CHECKPOINT — Exercise 3

Q1. Why use the with Session(engine) block?

04 Create & Read One Document

Goal: Create a new ORM object, save it, and retrieve one document by its primary key.

Steps

1. Build Document(...) from the validated DocumentCreate payload.
2. Use db.add(doc), then db.commit().
3. Use db.refresh(doc) to load database-generated values such as id.
4. Use db.get(Document, document_id) to read one document.
5. Return 404 when the requested document does not exist.

Starter / reference code

```
@app.post("/documents")
def create_document(payload: DocumentCreate):
    with Session(engine) as db:
        doc = Document(
            title=payload.title,
            content=payload.content,
            priority=payload.priority,
        )
        db.add(doc)
        db.commit()
        db.refresh(doc)
        return doc

@app.get("/documents/{document_id}")
def get_document(document_id: int):
    with Session(engine) as db:
        doc = db.get(Document, document_id)
        if doc is None:
            raise HTTPException(404, "Document not found")
        return doc
```

✓ CHECKPOINT — Exercise 4

Q1. Which call actually saves the new row?

05 Update, Delete & Verify Persistence

Goal: Complete CRUD and prove that the SQLite file really persists data.

Steps

1. GET a document with `db.get(Document, document_id)`.
2. PUT: change title, content and priority, then commit and refresh.
3. DELETE: call `db.delete(doc)`, then `db.commit()`.
4. Return 404 if the requested id does not exist.
5. Restart Uvicorn and GET a previously created document to prove persistence.

Starter / reference code

```
@app.put("/documents/{document_id}")
def update_document(document_id: int, payload: DocumentCreate):
    with Session(engine) as db:
        doc = db.get(Document, document_id)
        if doc is None:
            raise HTTPException(404, "Document not found")
        doc.title = payload.title
        doc.content = payload.content
        doc.priority = payload.priority
        db.commit()
        db.refresh(doc)
        return doc

@app.delete("/documents/{document_id}")
def delete_document(document_id: int):
    with Session(engine) as db:
        doc = db.get(Document, document_id)
        if doc is None:
            raise HTTPException(404, "Document not found")
        db.delete(doc)
        db.commit()
        return {"message": "Document deleted"}

# Persistence test: restart Uvicorn, then GET a document created before restart.
```

✓ CHECKPOINT — Exercise 5

Q1. How do you prove persistence is real?

06 Submission Checklist

- `documents.db` is created locally after the database is first used.
- Document ORM model maps id, title, content and priority.
- `Base.metadata.create_all(engine)` creates the missing table for this beginner lab.
- Every endpoint that touches the database uses `with Session(engine) as db:`.
- POST saves a row and returns the generated id.
- GET, PUT and DELETE work and return 404 when the id is missing.
- Previously created data is still available after restarting Uvicorn.